

AI-Powered Logistics Analytics Dashboard

Project Specification

Overview

Design, build, and deploy an AI-powered analytics dashboard for a logistics client.

This assignment evaluates your ability to build a full-stack application that handles structured data, delivers meaningful analytics, integrates AI responsibly, implements forecasting, ships to production, and communicates technical decisions with clarity.

Project Summary

Build a web application with two complementary interfaces: a traditional analytics dashboard showing KPIs and charts, and a natural-language interface powered by AI. Together they should support querying operational data, generating charts dynamically, answering business questions, and predicting demand.

Core Concept

The application must operate on one unified dataset and support three levels of intelligence.

Descriptive Analytics

Dashboards and visualizations that show what has happened.

Diagnostic Analytics

Natural-language queries answered directly from data — explaining why.

Predictive & Prescriptive Analytics

Forecasting future demand and recommending action.

Core Requirements

Dashboard

Create a dashboard displaying at minimum the following KPIs: total orders, delivered orders, delayed orders, on-time delivery rate, and average delivery time. Support at least two charts — for example, order volume over time, delivery performance (delayed vs on-time), and carrier or destination breakdown.

Natural Language Queries

Users must be able to ask questions such as "Show delayed orders by week for the last 3 months", "Which carrier has the highest delay rate?", or "How many orders were delivered late last month?". The system should interpret each question, retrieve the relevant data, and return a direct answer, a chart, or both.

Dynamic Chart Generation

The system must automatically select an appropriate chart type, render charts dynamically, and support a defined subset of analytical queries.

Explainability

Every answer or chart must be accompanied by the filters applied (e.g. time range), the metrics and dimensions used, a query plan or structured interpretation (recommended), and access to the underlying data as a table or summary.

Data Handling

Use the provided dataset or database. Treat all data as read-only and ensure correct aggregation and filtering throughout.

AI-Orchestrated Analytical Tools

The AI layer must act as a routing and orchestration system — not as the source of truth. AI should interpret the user's question, select the correct computation path, call the appropriate tool, and present results clearly. It must never generate answers without computation.

A. Query Tool (Analytics)

Used for dashboard queries, aggregations, and KPI calculations. Handles questions like "Show delayed orders by week" or "Which carrier has the highest delay rate?".

B. Forecasting Tool

Used for predicting future demand. Handles questions like "Predict demand for SKU X for the next 4 months" or "How much inventory should I plan?". The tool must use historical data from the dataset, apply a basic forecasting method, and return forecast values, a visualization of historical and forecast data, an inventory recommendation, and a methodology explanation. Acceptable methods include moving average, linear regression, exponential smoothing, and simple trend models.

Expected System Flow

User Question → AI Interpretation → Tool Selection → Structured Input → Computation
→ Result → Explanation → Visualization

Deployment Requirements

The application must be deployed to a publicly accessible URL, fully usable without local setup, and stable for reviewers. Any hosting platform is acceptable (e.g. Vercel, AWS). If authentication is used, provide test credentials. Do not commit secrets to the repository.

Technical Expectations

Any technology stack is acceptable. Common choices include React, Next.js, or Vue for the frontend; Node, Python, Java, or .NET for the backend; and PostgreSQL for the database.

Architecture Guidelines

Avoid executing raw AI-generated SQL without validation. Prefer structured query generation and clearly separate AI interpretation, data computation, and business logic.

Deliverables

Submit a source code repository, a live deployed application URL, and a README.md.

README Requirements

The README must cover: local setup instructions and environment variables; a system overview with key design decisions and data flow; an explanation of how questions are interpreted and tools are selected; assumptions, simplifications, limitations, and unsupported queries; and a section on future improvements.

Evaluation Criteria

Category	Weight
Product & UX	15%
Frontend	15%
Backend & Architecture	20%
Data Correctness	20%
AI Orchestration	15%
Forecasting	10%
Deployment	5%

Important Notes

Expected effort is 6–10 hours. We value clarity, correctness, and reasoning over completeness and polish. Prefer simple, correct solutions and explain your tradeoffs. Do not over-engineer. Undisclosed AI usage may be treated negatively.

Submission

Provide your repository link, the deployed app URL, and credentials if authentication is required.
